

4장 딥러닝 시작

🕒 생성일 @2025년 3월 17일 오후 1:17

1 인공 신경망의 한계와 딥러닝 출현

2 딥러닝 구조

1. 딥러닝 용어
2. 딥러닝 학습
3. 딥러닝의 문제점과 해결 방안
옵티마이저
4. 딥러닝을 사용할 때 이점

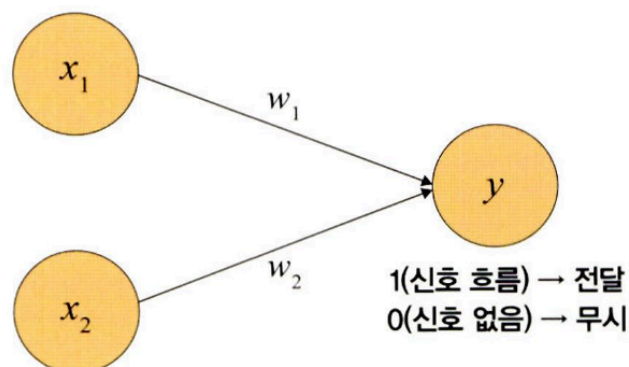
3 딥러닝 알고리즘

1. 심층 신경망 (DNN)
2. 합성곱 신경망 (CNN)
3. 순환 신경망 (RNN)
4. 제한된 볼츠만 머신
5. 심층 신뢰 신경망 (DBN)

4 우리는 무엇을 배워야 할까?

1 인공 신경망의 한계와 딥러닝 출현

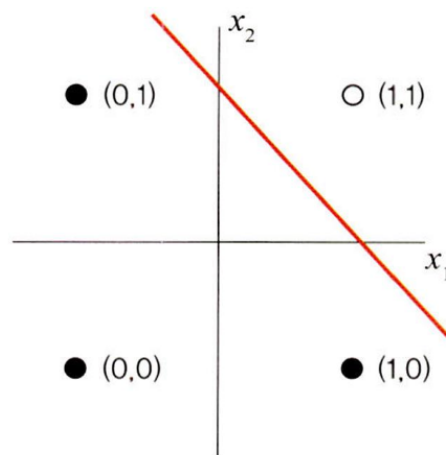
- 퍼셉트론
 - 오늘날 신경망(딥러닝)의 기원이 되는 알고리즘
 - 입력층, 출력층, 가중치로 구성된 구조의 선형 분류기
 - 다수의 신호 (흐름이 있는)를 입력으로 받아 하나의 신호를 출력 → 흐른다/안흐른다 (=1/0) 정보를 앞으로 전달하는 원리로 작동



- 논리 게이트
 1. AND 게이트

- 모든 입력이 1일때 작동
- 어떤 하나라도 0을 갖는다면 작동을 멈춤

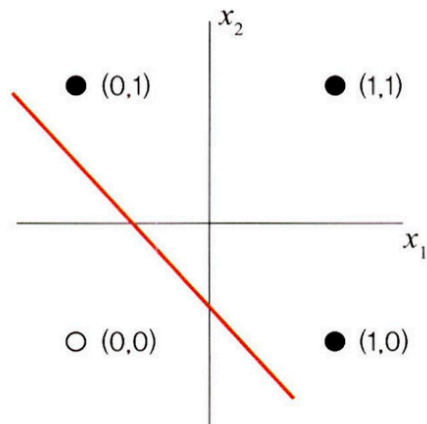
x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1



2. OR 게이트

- 둘 중 하나만 1 / 둘 다 1일때 작동
- 입력모두가 0을 갖는 경우를 제외한 경우

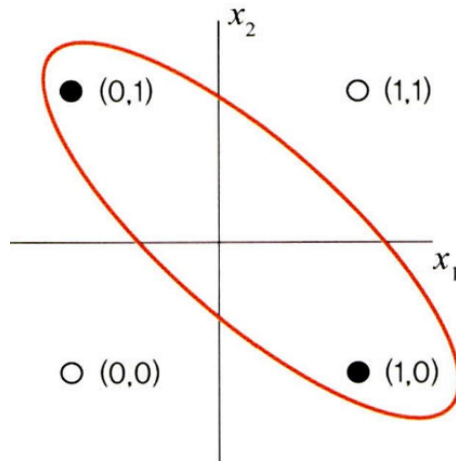
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1



3. XOR 게이트

- 배타적 논리합
- 둘 중 한 개만 1일 때 작동
- 데이터가 비선형적으로 분리 → 제대로 된 분류가 어려움
- 단층 퍼셉트론에서 AND, OR 연산 학습 가능 but, XOR 학습 불가능
 - 이를 극복하는 방안으로 입력층·출력층 사이에 하나 이상의 중간층(은닉층)을 두어 비선형적으로 분리되는 데이터에 대해서도 학습이 가능한 **다층 퍼셉트론** 고안
 - ⇒ 입력층과 출력층 사이에 은닉층이 여러 개 있는 신경망 = 심층 신경망 = 딥러닝

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

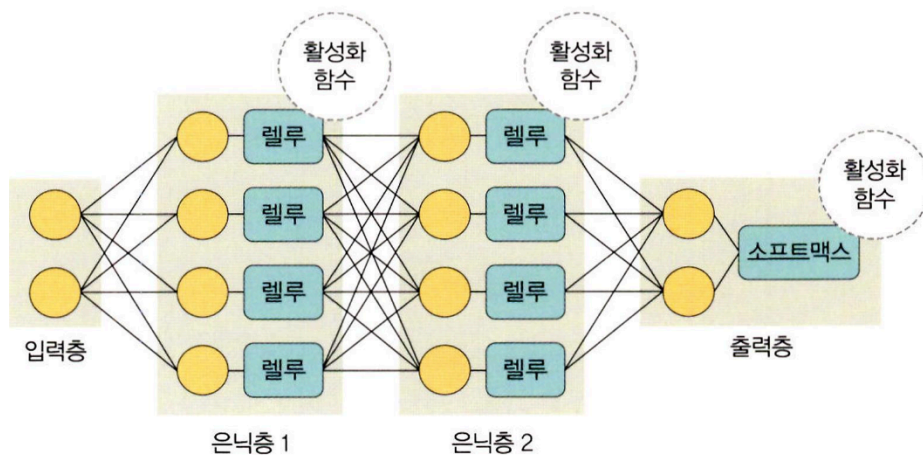


2 딥러닝 구조

- 딥러닝 : 여러 층을 가진 인공 신경망을 사용하여 학습을 수행하는 것

1. 딥러닝 용어

- 딥러닝 = 입력층 + 출력층 + 두 개 이상의 은닉층 (+ 입력 신호 전달 위한 다양한 함수)



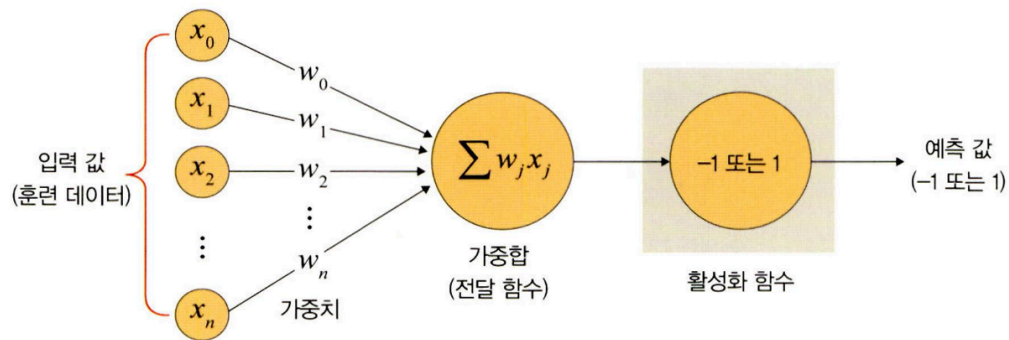
- 딥러닝 구성요소

구분	구성 요소	설명
층	입력층(input layer)	데이터를 받아들이는 층
	은닉층(hidden layer)	모든 입력 노드부터 입력 값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
	출력층(output layer)	신경망의 최종 결괏값이 포함된 층

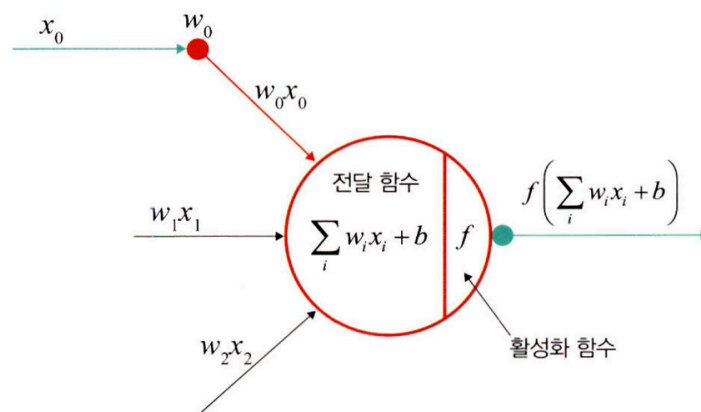
구분	구성 요소	설명
가중치(weight)		노드와 노드 간 연결 강도
바이어스(bias)		가중합에 더해 주는 상수로, 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할을 함
가중합(weighted sum), 전달 함수		가중치와 신호의 곱을 합한 것
함수	활성화 함수(activation function)	신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
	손실 함수(loss function)	가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

- **가중치:** 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소

ex. w_1 값이 0 혹은 0과 가까운 0.001이라면 x_1 이 아무리 큰 값이라도 $x_1 * w_1$ 값은 0이거나 0에 가까운 값이 됨. → 이렇게 입력 값의 연산 결과를 조정하는 역할을 하는 것이 가중치



- **가중합 / 전달 함수**



- 각 노드에서 들어오는 신호에 가중치를 곱함 → 다음 노드로 전달됨 → 이 값들을 모두 더한 합계 = 가중합
- 노드의 가중합이 계산됨 → 이 가중합을 활성화 함수로 보냄 = 전달 함수라고도 함

◦ 가중합 공식

$$\sum_i w_i x_i + b$$

(w : 가중치, b : 바이어스)

• 활성화 함수

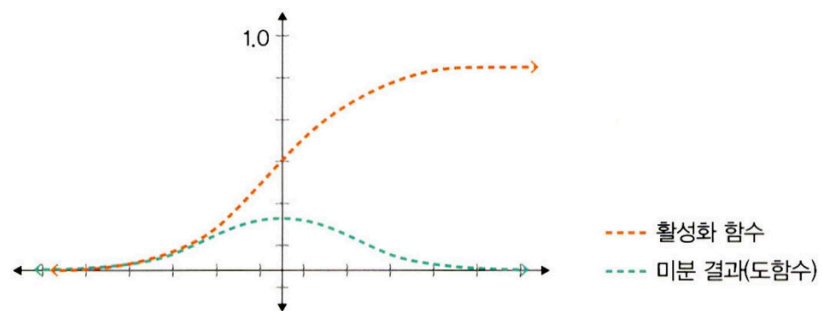
: 전달 함수에서 전달받은 값을 출력할 때, 일정기준에 따라 출력 값을 변화시키는 비선형 함수

a. 시그모이드 함수

: 선형 함수의 결과를 0 ~ 1 사이에서 비선형 형태로 변형해줌

- 로지스스틱 회귀와 같은 분류 문제를 확률적으로 표현하는 데 사용됨
- 과거에는 인기 ↑ but 딥러닝 모델의 깊이가 깊어지면 기울기가 사라지는 '기울기 소멸 문제' 발생 → 딥러닝 모델에서 잘 사용 x
- 수식

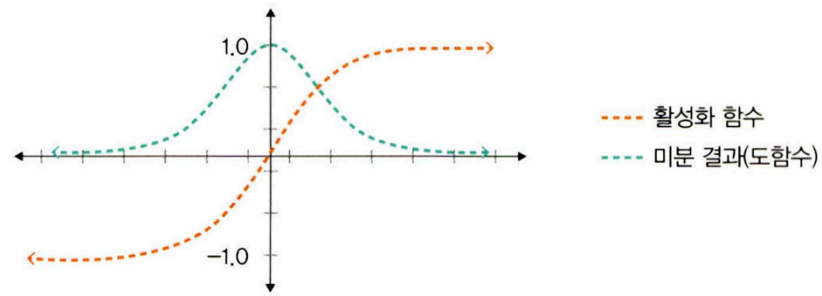
$$f(x) = \frac{1}{1 + e^{-x}}$$



b. 하이퍼볼릭 탄젠트 함수

: 선형 함수의 결과를 -1 ~ 1 사이에서 비선형 형태로 변형해줌

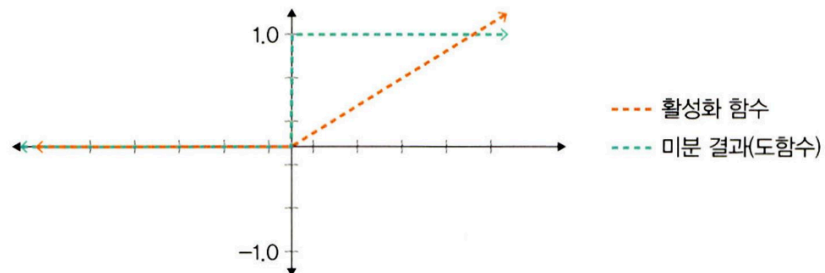
- 시그모이드에서 결괏값의 평균이 0이 아닌 양수로 편향된 문제를 해결하는 데 사용 but 기울기 소멸 문제 여전히 발생



c. 렐루 함수

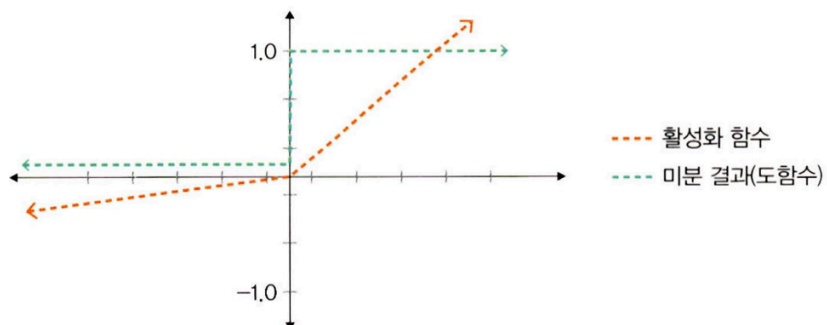
: 입력이 음수 $\rightarrow 0$ 출력, 양수 $\rightarrow x$ 출력

- 최근 활발히 사용됨
- 경사 하강법에 영향을 주지 않아 학습 속도가 빠르고, 기울기 소멸 문제가 발생하지 않는다는 장점이 있음
- 일반적으로 은닉층에서 사용됨
- 하이퍼볼릭 탄젠트 함수 대비 학습 속도가 6배 빠름
- BUT 음수 값을 받으면 항상 0을 출력 $\rightarrow$ 학습 능력 $\downarrow$
 $\Rightarrow$ 해결위해 리키 렐루 함수 등 사용



d. 리키 렐루 함수

- 입력 값이 음수이면 0이 아닌 0.001 처럼 매우 작은 수를 반환
 $\rightarrow$ 입력 값이 수렴하는 구간 제거 $\rightarrow$ 렐루 함수 문제 해결



e. 소프트맥스 함수

- 입력 값을 0 ~ 1 사이에 출력되도록 정규화 → 출력 값들의 총합이 항상 1이 되도록 함
- 딥러닝에서 출력 노드의 활성화 함수로 많이 사용됨
- 수식

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

- $\exp(x)$ 는 지수함수를 의미
- n 은 출력층 뉴런 개수
- y_k 는 그 중 k 번째 출력

⇒ 함수의 분자 = 입력 신호 a_k 의 지수 함수 / 분모 = 모든 입력 신호의 지수 함수 합

- 손실 함수

- 경사 하강법: 학습률과 손실 함수의 순간 기울기를 이용하여 가중치를 업데이트 하는 방법
= 미분의 기울기를 이용하여 오차를 비교 → 최소화하는 방향으로 이동 시키는 방법
⇒ 오차를 구하는 방법이 손실 함수
- : 학습을 통해 얻은 데이터 추정치가 실제 데이터와 얼마나 차이 나는지 평가하는 지표
→ 값이 클수록 많이 틀렸다는 의미, 0에 가까우면 완벽하게 추정할 수 있다는 의미

a. 평균 제곱 오차 (MSE)

: 실제 값과 예측 값의 차이를 제곱하여 평균을 낸 것

→ 실제 값 - 예측값 차이 ↑ ⇒ 평균 제곱 오차 값 ↑

= 평균 제곱 오차 값 ↓ ⇒ 예측력 ↑

- 회귀에서 손실 함수로 주로 사용
- 수식

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$$

b. 크로스 엔트로피 오차 (CEE)

: 분류 문제에서 원-핫 인코딩했을 때만 사용할 수 있는 오차 계산법

- 일반 분류 문제에서는 데이터의 출력을 0과 1로 구분하기 위해 시그모이드 함수 사용

but 시그모이드 함수에 포함된 자연 상수 e 때문에 평균 제곱 오차를 적용하면 매끄럽지 못한 그래프가 출력됨

∴ 크로스 엔트로피 손실 함수 사용

but 경사 하강법 과정에서 학습이 지역 최소점에서 멈출 수 있음

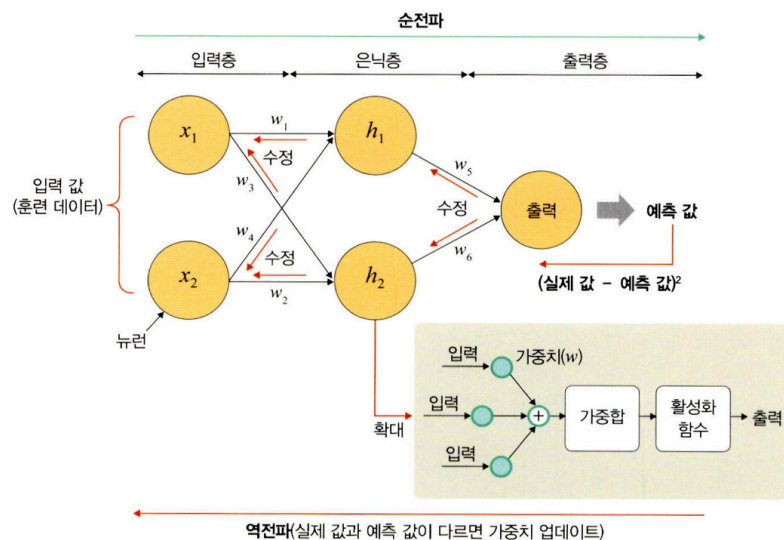
→ 이를 방지하고자 자연상수 e에 반대되는 자연로그를 모델의 출력 값에 취함

- 수식

$$CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$$

$$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$$

2. 딥러닝 학습



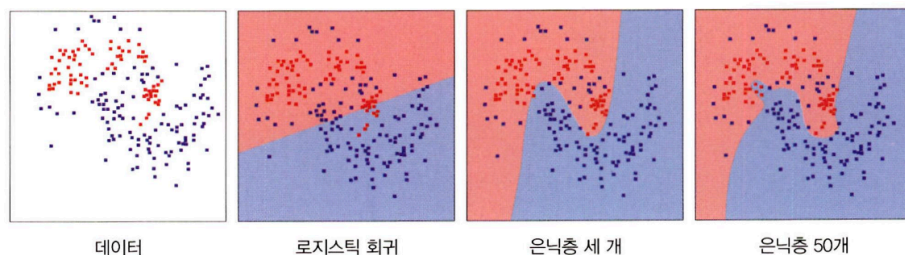
1. 순전파 (입력층 → 은닉층 → 출력층)

- 네트워크 훈련 데이터가 들어올 때 발생
- 데이터를 기반으로 예측 값을 계산하기 위해 전체 신경망을 교차해 지나감
- 즉, 모든 뉴런이 이전 층의 뉴런에서 수신한 정보에 변환 (가중합 및 활성화 함수)을 적용 → 다음 층(은닉층)의 뉴런으로 전송하는 방식
- 네트워크를 통해 입력 데이터를 전달하며, 데이터가 모든 층을 통과하고 모든 뉴런이 계산을 완료하면 그 예측 값은 최종 층(출력층)에 도달하게 됨

2. 역전파 (출력층 → 은닉층 → 입력층)

- 손실 함수로 네트워크의 예측 값과 실제 값의 차이(손실, 오차)를 추정
→ 손실 함수 비용은 '0'이 이상적임
∴ 손실 함수 비용이 0에 가깝도록 하기 위해 모델이 훈련을 반복하면서 가중치 조정
- 손실(오차)이 계산되면 그 정보는 역으로 전파됨
- 출력층에서 시작된 손실 비용은 은닉층의 모든 뉴런으로 전파되지만, 은닉층의 뉴런은 각 뉴런이 원래 출력에 기여한 상대적 기여도(=가중치)에 따라 값이 달라짐
- 예측 값과 실제 값의 차이를 각 뉴런의 가중치로 미분한 후 기존 가중치 값에서 뺌
이 과정을 출력층→은닉층→입력층 순서로 모든 뉴런에 대해 진행하여 계산된 각 뉴런 결과를 또다시 순전파의 가중치 값으로 사용함

3. 딥러닝의 문제점과 해결 방안

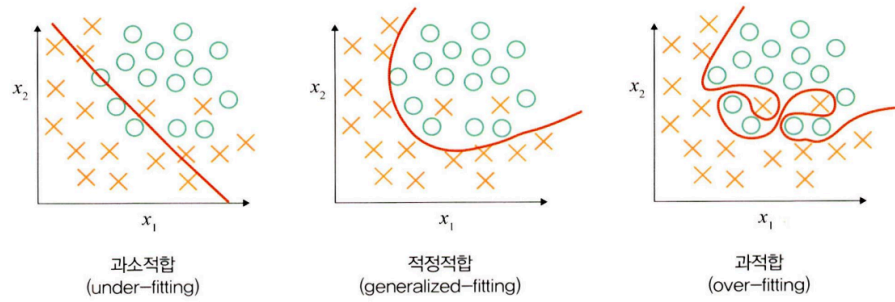


딥러닝의 핵심은 활성화 함수가 적용된 여러 은닉층을 결합하여 비선형 영역을 표현하는 것

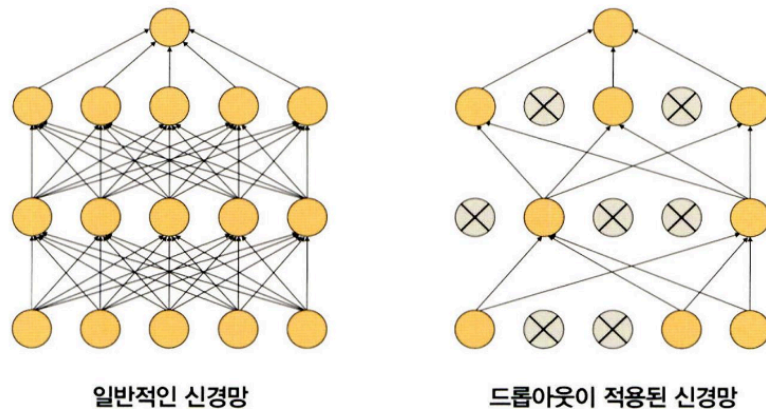
→ 활성화 함수가 적용된 은닉층 개수가 많을수록 데이터 분류가 잘됨

but 은닉층이 많을수록 다음 세 가지 문제점 발생

1. 과적합 문제 발생

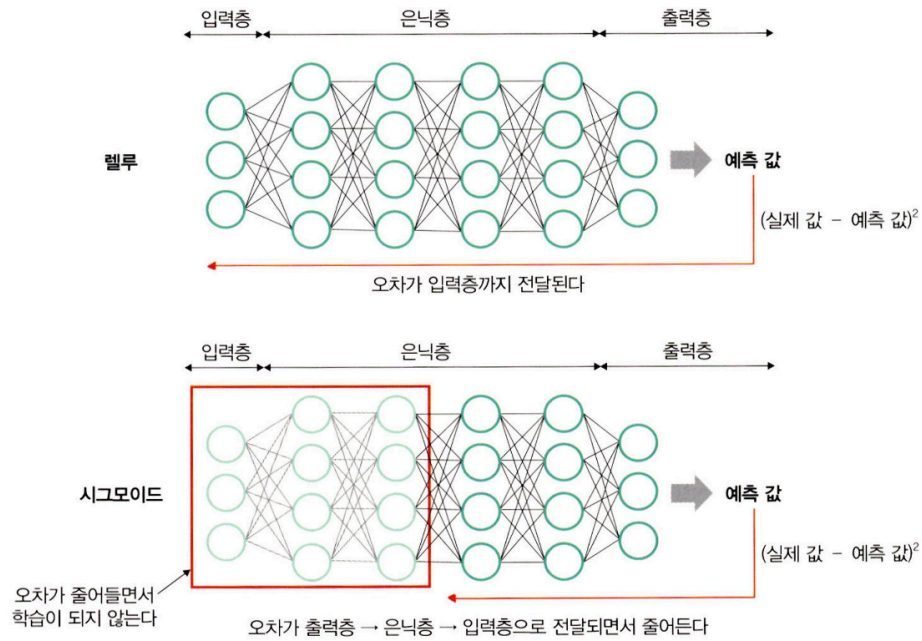


- 훈련 데이터를 과하게 학습해서 발생함
- 일반 훈련데이터는 실제 데이터의 일부분임
 $\therefore$ 훈련 데이터를 과하게 학습했기 때문에 예측 값과 실제 값 차이인 오차가 감소하지만, 검증 데이터에 대해서는 오차가 증가할 수 있음
 $\rightarrow$ 과적합은 훈련 데이터에 대해 과하게 학습하여 실제 데이터에 대한 오차가 증가하는 현상을 의미함
- 과적합 해결방법 : 드롭아웃



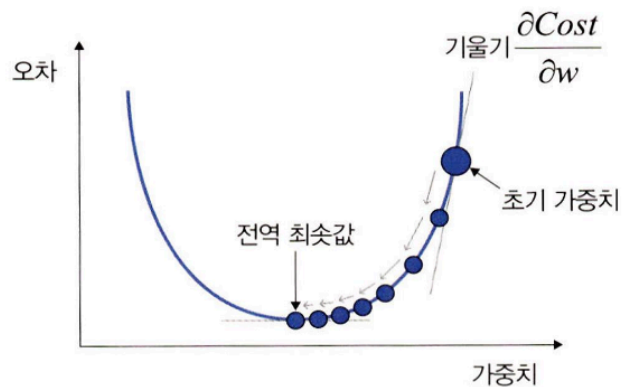
- 학습 과정 중 임의로 일부 노드들을 학습에서 제외시킴

2. 기울기 소멸 문제 발생

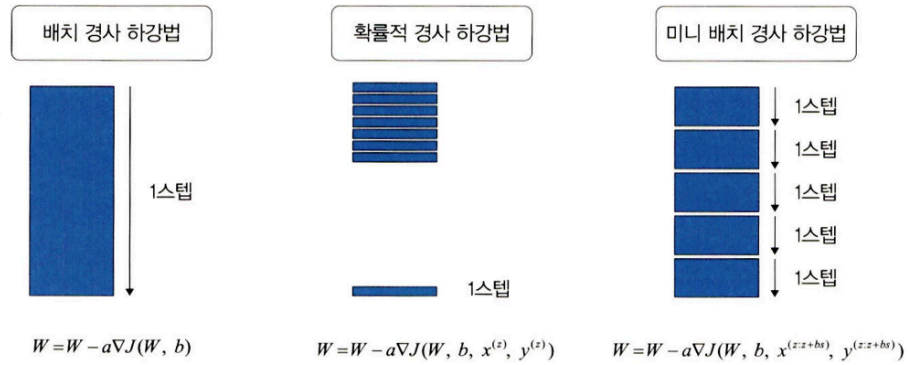


- 은닉층이 많은 신경망에서 주로 발생하는데, (역전파 中) 출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상
- 기울기가 소멸되기 때문에 학습되는 양이 0에 가까워져 학습이 더디게 진행된다. 오차를 더 줄이지 못하고 그 상태로 수렴하는 현상
- 시그모이드·하이퍼볼릭 탄젠트 대신 렐루 활성화 함수를 사용하면 해결 가능

3. 성능이 나빠지는 문제 발생



- 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정을 반복하는데, 이때 성능이 나빠지는 문제가 발생함
→ 이를 해결하고자 확률적 경사 하강법 / 미니 배치 하강법 사용



○ 배치 경사 하강법 (BGD)

: 전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산하여 모델의 파라미터를 업데이트 하는 방법, 전체 훈련 데이터셋에 대해 가중치를 편미분하는 방법

■ 수식

손실 함수의 값을 최소화하기 위해 기울기(∇) 이용

$$W = W - a \nabla J(W, b)$$

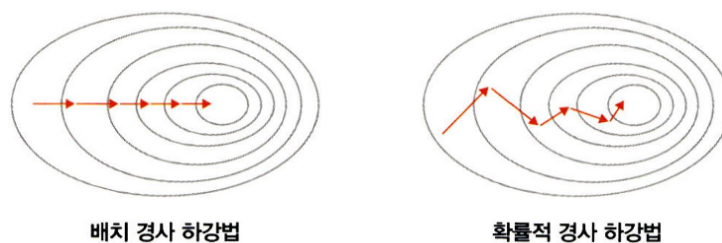
(a : 학습률, J : 손실 함수)

- 한 스텝에 모든 훈련 데이터셋을 사용하므로 학습이 오래 걸리는 단점 있음
→이를 개선한 것이 확률적 경사 하강법

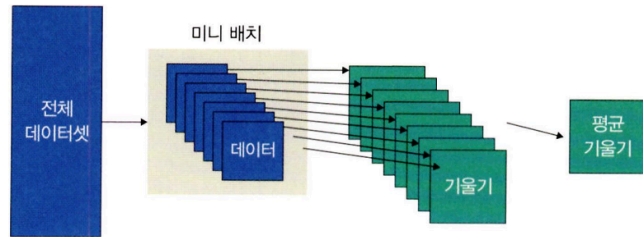
○ 확률적 경사 하강법 (SGD)

: 임의로 선택한 데이터에 대해 기울기를 계산하는 방법

- 적은 데이터를 사용하므로 빠른 계산 가능
- (이미지 참고) 파라미터 변경 폭이 불안정하고 때로는 배치 경사 하강법보다 정확도가 낮을 수 있지만, 속도가 빠르다는 장점이 있음

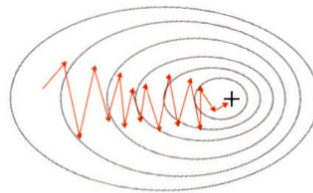


○ 미니 배치 경사 하강법

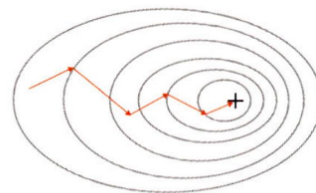


: 전체 데이터셋을 미니 배치 여러개로 나누고, 미니 배치 한 개마다 기울기를 구한 후 그것의 평균 기울기를 이용하여 모델을 업데이트해서 학습하는 방법

- 전체 데이터를 계산하는 것보다 빠르며, 확률적 경사 하강법보다 안정적이라는 장점이 있음 → 실제로 가장 많이 사용함
- (이미지 참고) 파라미터 병경 폭이 확률적 경사 하강법에 비해 안정적이면서 속도도 빠름



확률적 경사 하강법

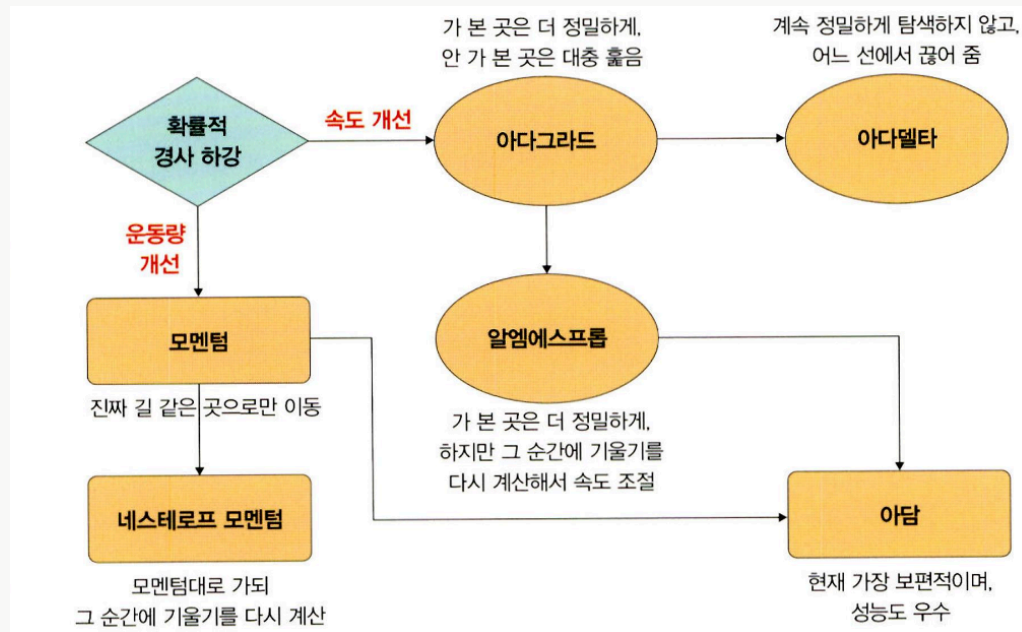


미니 배치 경사 하강법



옵티마이저

확률적 경사 하강법의 파라미터 변경 폭이 불안정한 문제를 해결하기 위해 학습속도와 운동량을 조정하는 옵티마이저를 적용할 수 있음



- 속도 조정 방법

- 아다그라드

- 변수(가중치)의 업데이트 횟수에 따라 학습률을 조정하는 방법,
- 많이 변화하지 않는 변수들의 학습률은 크게하고, 많이 변화하는 변수들의 학습률을 작게 함
- 많이 변화한 변수는 최적 값에 근접했을 것이라는 가정하에 작은 크기로 이하면서 세밀하게 값을 조정 ↔ 적게 변화한 변수들은 학습률을 크게 하여 빠르게 오차 값을 줄이고자 하는 방법
- 수식

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$

$$G(i) = G(i-1) + (\nabla E(w(i)))^2$$

- 파라미터마다 다른 학습률을 주기 위해 G함수 추가
- G 값은 이전 G 값의 누적 (기울기 크기의 누적)

- 기울기가 크면 G값이 커지기 때문에 학습률 n은 작아짐

⇒ 파라미터 학습 ↑ 작은 학습률로 업데이트, 파라미터 학습 ↓ 높은 학습률로 업데이트 됨

```
optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01)
```

⇒ 아다그라드는 기울기가 0에 수렴하는 문제가 있어 사용하지 X, 대신 알엠에스프 사용

- 아다델타

$$w(i+1) = w(i) - \frac{\sqrt{D(i-1)+\epsilon}}{\sqrt{G(i)+\epsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

$$D(i) = \gamma D(i-1) + (1-\gamma)(\Delta(w(i)))^2$$

- 아다그라드에서 G 값이 커짐에 따라 학습이 멈추는 문제를 해결하기 위해 장한 방법
- 아다델타에는 아다그라드의 수식에서 학습률n을 D함수(가중치의 변화량의 제곱을 누적한 값)로 변환했기 때문에 학습률에 대한 하이퍼파라미터가 필요하지 않음

```
optimizer = torch.optim.Adadelta(model.parameters(), lr=1.0)
```

- 알엠에스프

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\epsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

- 아다그라드의 G(i)값이 무한히 커지는 것을 방지하고자 제안된 방법
- G 함수에서 감마만 추가됨
 - 즉, G 값이 너무 크면 학습률이 작아져 학습이 안 될 수 있으므로 사용지 감마 값을 이용하여 학습률 크기를 비율로 조정

```
optimizer = torch.optim.RMSprop(model.parameters(), lr=0.01)
```

- 운동량 조정 방법

- 모멘텀

- 경사 하강법과 마찬가지로 매번 기울기를 구하지만, 가중치를 수정하기 전에 이전 수정 방향(+, -)을 참고하여 같은 방향으로 일정한 비율만 수정하는 방식
 - 수정이 양(+)의 방향과 음(-)의 방향으로 순차적으로 일어나는 지그재그 현상이 줄어들고, 이전 이동값을 고려하여 일정 비율만큼 다음값을 결정하므로 성효과를 얻을 수 있는 장점
 - 모멘텀은 SGD(확률적 경사 하강법)와 함께 사용
 - 확률적 경사 하강법 수식

$$w(i+1) = w(i) - \eta \nabla E(w(i))$$

- 모멘텀 수식

$$\begin{aligned} w(i+1) &= w(i) - v(i) \\ v(i) &= \gamma v(i-1) + \eta \nabla E(w(i)) \end{aligned}$$

- 기울기 크기와 반대 방향만큼 가중치를 업데이트 → 기울기가 크면 아래쪽(-) 방향으로 업데이트함
 - SGD 모멘텀은 확률적 경사 하강법에서 기울기를 속도로 대체하여 사용하는 방식으로 이전 속도의 일정 부분 반영
 - 이전에 학습했던 속도와 현재 기울기를 반영하여 가중치를 구함

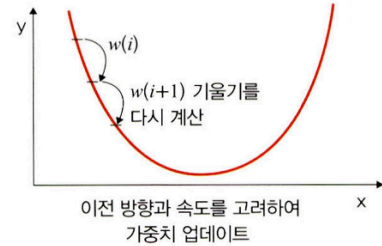
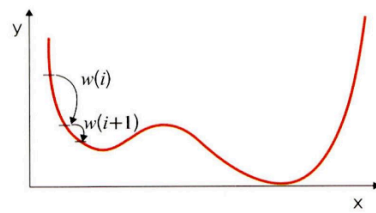
```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

- 네스테로프 모멘텀

$$\begin{aligned} w(i+1) &= w(i) - v(i) \\ v(i) &= \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1)) \end{aligned}$$

- 모멘텀 값과 기울기 값이 더해져 실제 값을 만드는 기존 모멘텀과 달리 모멘텀 값이 적용된 지점에서 기울기 값을 계산
 - 모멘텀 방법은 멈추어야 할 시점에서도 관성에 의해 훨씬 멀리 갈 수 있는 문제점 ↔ 네스테로프 방법은 모멘텀으로 절반 정도 이동한 후 어떤 방식으로 멈춰야 하는지 다시 계산하여 결정하기 때문에 모멘텀 방법의 단점을 극복

∴ 모멘텀 방법의 이점인 빠름이동속도는 그대로 가져가면서 멈추어야 할 절한 시점에서 제동을 거는 데 훨씬 용이



```
optimizer = torch.optim. SGD(model.parameters(), 1r=0.01, mo
```

- 속도와 운동량에 대한 혼용 방법
 - 아담

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\epsilon}} v(i)$$

$$G(i) = \gamma_2 G(i-1) + (1-\gamma_2)(\nabla E(w(i)))^2$$

$$v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$$

: 모멘텀과 알엠에스프롭의 장점을 결합한 경사하강법

```
optimizer = torch.optim.Adam(model.parameters(),1r=0.01)
```

4. 딥러닝을 사용할 때 이점

- 특성 추출

: 컴퓨터가 입력 받은 데이터를 분석 → 일정한 패턴이나 규칙을 찾아내려면 사람이 인지하는 데이터를 컴퓨터가 인지할 수 있는 데이터로 변환해주어야 됨

⇒ 이때 데이터별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환하는 작업

- 딥러닝 활성화 이전 많이 사용되었던 머신러닝 알고리즘 SVM, 나이브 베이즈, 로지스틱 회귀의 특성 추출 → 매우 복잡, 수집된 데이터에 대한 전문 지식 필요

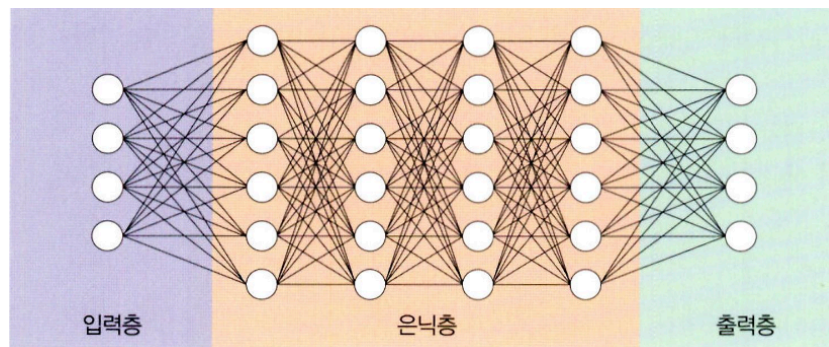
but 딥러닝에서는 이러한 특성 추출 과정을 알고리즘에 통합시킴

∴ 데이터 특성을 잘 잡아내고자 은닉층을 깊게 쌓는 방식으로 파라미터를 늘린 모델 구조 덕분

- 빅데이터의 효율적 활용
 - 특성 추출을 알고리즘에 통합하는 것이 가능하게 만든 것은 빅데이터임.
 - 확보된 데이터의 양이 적다면 딥러닝 성능향상을 기대하기 힘들 ⇒ 머신 러닝 고려해야

3 딥러닝 알고리즘

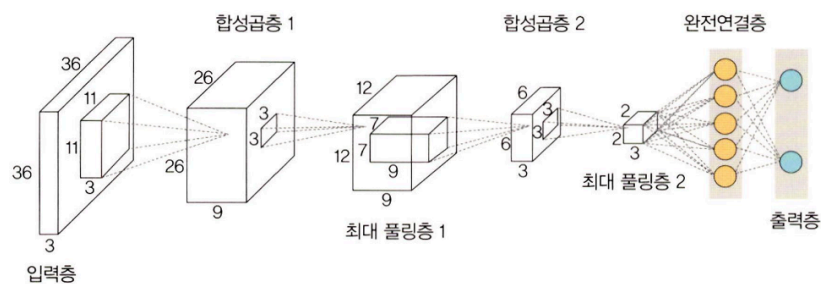
1. 심층 신경망 (DNN)



: 입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망

- 다수의 은닉층 추가 → 별도 트릭 없이 비선형 분류 가능 (↔ 머신 러닝)
- 학습을 위한 연산량이 많고, 기울기 소멸 문제 등이 발생할 수 있음
- 드롭 아웃, 렐루 함수, 배치 정규화 등 적용 필요

2. 합성곱 신경망 (CNN)



: 합성곱층과 풀링층을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘.

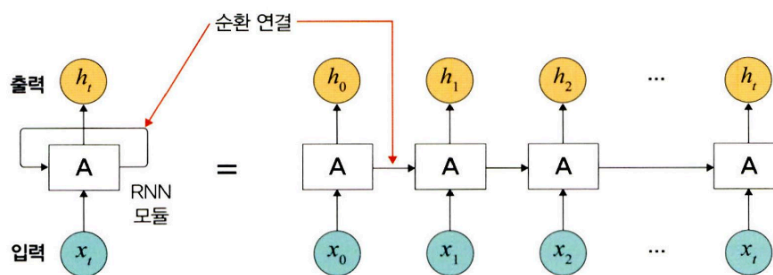
- 영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용한 신경망
- 대표적 CNN : LeNet-5, AlexNet / 층이 더 깊은 CNN: VGG, GoogLeNet, ResNet
- 기존 신경망과의 차별점

- 각층의 입출력 형상을 유지
- 이미지의 공간 정보를 유지하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식
- 복수 필터로 이미지의 특징을 추출하고 학습
- 추출한 이미지의 특징을 모으고 강화하는 풀링층
- 필터를 공유 파라미터로 사용하기 때문에 일반 인공신경망과 비교하여 학습 파라미터가 매우 적음

3. 순환 신경망 (RNN)

: 시계열 데이터 (음악·영상 등) 같이 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망

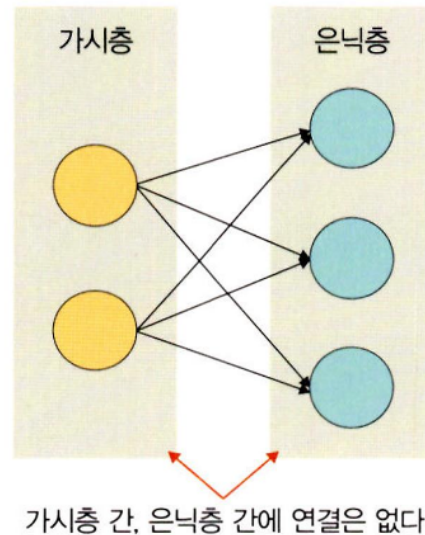
→ 순환 = 자기 자신을 참조한다는 것, 현재 결과가 이전 결과와 연관 있다는 의미



- 특징
 - 시간성을 가진 데이터가 많음
 - 시간성 정보를 이용하여 데이터의 특징을 잘 다룸
 - 시간에 따라 내용이 변하므로 데이터는 동적이고, 길이가 가변적
 - 매우 긴 데이터를 처리하는 연구가 활발히 진행되고 있음
- 기울기 소멸 문제로 학습이 제대로 되지 않는 문제가 있음
 - 이를 해결하고자 메모리 개념을 도입한 LSTM이 순환 신경망에서 많이 사용됨
- 순환 신경망 - 자연어 처리 분야와 시너지
 - ex. 언어 모델링, 텍스트 생성, 자동 번역(기계 번역), 음성 인식, 이미지 캡션 등

4. 제한된 볼츠만 머신

: 가시층과 은닉층으로 구성된 모델, 가시층은 은닉층과만 연결

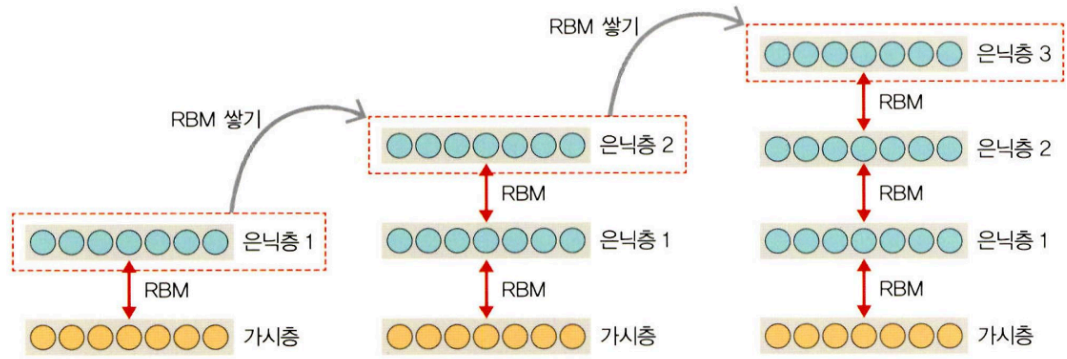


- 특징
 - 차원 감소, 분류, 선형 회귀 분석, 협업 필터링, 특성 값 학습, 주제 모델링에 사용
 - 기울기 소멸 문제를 해결하기 위해 사전 학습용으로 활용가능
 - 심층신뢰신경망(DBN)의 요소로 활용

5. 심층 신뢰 신경망 (DBN)

: 입력층과 은닉층으로 구성된 제한된 볼츠만 머신을 여러 층으로 쌓은 형태로 연결된 신경망

- 사전 훈련된 제한된볼츠만머신을 쌓아올린 구조
- 레이블이 없는 데이터에 대한 비지도 학습 가능
- 이미지에서 전체를 연상하는 일반화·추상화 과정을 구현할 때 유용
- 학습절차
 1. 가시층과 은닉층 1에 제한된 볼츠만 머신을 사전훈련
 2. 첫 번째 층 입력 데이터와 파라미터를 고정하여 두 번째 층 제한된 볼츠만 머신을 사전 훈련
 3. 원하는 층 개수만큼 제한된 볼츠만 머신을 쌓아 올려 전체 DBN을 완성



- 특징
 - 순차적으로 심층 신경망을 학습시켜 가면서 계층적 구조를 생성
 - 비지도 학습으로 학습
 - 위로 올라갈수록 추상적 특성을 추출
 - 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용

4 우리는 무엇을 배워야 할까?

(생략)